Project Title: KubePilot — Predictive & Reactive Autoscaling in Kubernetes (Project 357)
Institution: Alliance University, Department of Computer Science & Engineering
Audit Scope: Verification of Existing Implementation against the Experimentation Specification
Environment Inspected: Minikube (Docker driver `192.168.49.2`), Kubernetes v1.32.0, Python 3.13.14 on Windows 10/11
Target Workload: `sample-web-app` (Deployment / Service in namespace `autoscaling-demo`)
Monitoring: `prometheus-server` (Deployment / Service in namespace `monitoring`)
Audit Date: October 24, 2026

---

1. Executive Summary

This report provides an evidence-based audit of Project 357 against the project's **Experimentation** slide requirements. The audit evaluates code existence, automated test coverage, and actual live runtime execution across three core pillars:

1. **Experimental Setup** (Native HPA baseline, proposed KubePilot framework, Prometheus monitoring, workload generators).
2. **Workload Patterns** (Stable, gradual increase, sudden spike, bursty workload, sudden drop).
3. **Experimental Procedure** (Comparable configuration, metric recording, repetition, comparative analysis).

Key Finding

The software architectural foundation—including the sample application, Prometheus telemetry pipeline, PRAH policy engine, safety guardrails, dry-run decision logger, and web dashboard—is **implemented and verified by 86 unit and regression tests**. However, **experimental workload execution has not yet been conducted**. No native HPA manifest exists in the repository, neither `k6` nor `locust` is installed on the host, no traffic generator scripts exist, and no comparative performance data has been gathered against the live cluster.

---

2. Experimental Setup Audit

2.1 Baseline: Kubernetes Native Horizontal Pod Autoscaler (HPA)

- **Code & Configuration: Does not exist.** No HorizontalPodAutoscaler YAML manifests exist in `k8s/` or anywhere in the repository.
- **Tested: No.** Zero automated baseline tests exist for native HPA.
- **Runtime Status: Not running.** Verified live via `kubectl get hpa -A` which returned `No resources found`.
- **Infrastructure Prerequisite:** The Kubernetes `metrics-server` is running in `kube-system` (`metrics-server-9d74bb658-sqxgw`, `1/1 Running`), and `kubectl top nodes` / `kubectl top pods` function properly. The cluster infrastructure supports HPA, but the baseline workload scaling policy has never been applied.

2.2 Proposed System: Policy-Based Scaling Coordination Framework (KubePilot)

- **Code & Configuration: Exists.** The `controller/` package contains complete implementations of:
  - `policy_engine.py` (Multi-metric threshold, hysteresis, and cooldown evaluation).
  - `strategy_coordinator.py` (PRAH priority arbitration and native HPA collision detection).
  - `guardrails.py` (Min/max replica clamps, maximum step-change boundaries).
  - `kubernetes_executor.py` (Kubernetes Deployment API patch and replica verification).
  - `decision_logger.py` (JSONL audit logging of all evaluation decisions).
  - Five active policies configured in `policies/scaling-policies.yaml`.
- **Tested: Yes (offline & mocked).** All 86 unit and regression tests pass (`python -m pytest tests/ -v`).
- **Runtime Status: Partially verified in dry-run mode.** The controller runs locally via `python -m controller.main`, connects to Minikube, and polls live Prometheus. However, closed-loop live scaling (`DRY_RUN=false`) responding

dynamically to workload pressure has **not yet been executed or verified** against real traffic in the live cluster.

2.3 Monitoring Infrastructure

- **Prometheus: Running and verified.** `service/prometheus-server` is running in namespace `monitoring` (port 9090). Scrapes live cAdvisor container metrics and application-level metrics (`/metrics`).
- **Kubernetes Events: Partially implemented.** Read RBAC permissions exist in `k8s/controller-rbac.yaml` and the dashboard queries events, but automated event export/recording during benchmarks is not yet scripted.
- **Application Logs: Available.** Live logs can be streamed via `kubectl logs -n autoscaling-demo -l app=sample-web-app`.

2.4 Workload Generator (k6 or Locust)

- **Code & Configuration: Does not exist.** No k6 scripts (`*.js`), Locust test files (`locustfile.py`), or load test harnesses exist in the repository.
- **Tested: No.**
- **Runtime Status:** Neither `k6` (`where.exe k6` failed) nor `locust` (`ModuleNotFoundError: No module named 'locust'`) is installed on the host system.

---

3. Workload Patterns Audit

Workload Pattern	Generator Script Exists?	Executed Against Cluster?	Telemetry Collected?	Scaling Behavior Verified?
1. Stable Workload *(Constant request rate)*	**No**	**No**	No live experiment telemetry	No
2. Gradual Increase *(Steadily rising demand)*	**No**	**No**	No live experiment telemetry	No
3. Sudden Spike *(Rapid increase in requests)*	**No** *(only mocked in unit tests & UI scenario)*	**No**	No live experiment telemetry	No
4. Bursty Workload *(Repeated fluctuations)*	**No**	**No**	No live experiment telemetry	No
5. Sudden Drop *(Rapid reduction in demand)*	**No**	**No**	No live experiment telemetry	No

Note on mock scenarios: While the UI contains buttons to inject synthetic metrics (`HIGH_CPU`, `CRITICAL_CPU`) into an in-memory dictionary via `/api/controls/simulate-metric`, these are frontend demo simulations and **not** real HTTP traffic generated against the workload.

---

4. Experimental Procedure Audit

- **Comparable Configuration (Baseline HPA vs KubePilot): Not implemented.** No specification exists matching KubePilot's thresholds (e.g., CPU > 75%) with an HPA `targetCPUUtilizationPercentage` manifest.
- **Run Each Workload & Record Metrics: Not implemented.** No experiment runner script exists to capture latency, CPU utilization, and replica count over time.
- **Repeat Tests Under Identical Configurations: Not implemented.** No automated test harness or reproducible seeds exist.
- **Compare Performance, Resource Utilization & Stability: Not implemented.** No comparative CSVs, Pandas dataframes, or visualization graphs exist.

---

5. Comprehensive Audit Table

Experiment item	Status	Evidence found	Missing work	Exact next action
Baseline: Native HPA Manifest	**NOT IMPLEMENTED**	`metrics-server` is running in `kube-system`; `kubectl top pods` works. `kubectl get hpa -A` returned 0 resources. No HPA YAML in repo.	Create an HPA manifest targeting `sample-web-app` (min: 2, max: 10, targetCPU: 75%).	Author `k8s/baseline-hpa.yaml` matching KubePilot CPU thresholds.
Baseline: HPA Workload Execution	**NOT IMPLEMENTED**	None. Zero baseline runs logged.	Apply HPA manifest, run workload generator, and record replica scaling and convergence time.	Execute baseline load test under all 5 workload

patterns. |
Proposed System: PRAH Decision Controller	**IMPLEMENTED BUT NOT VERIFIED**	Complete codebase in `controller/`; 86/86 unit/regression tests pass; decision loop logs dry-run entries to `logs/decisions.jsonl`.	Live scaling validation under external load with `DRY_RUN=false`.	Run `controller/main.py` in live mode during real workload traffic and verify replica mutation via `kubectl get deployment`.
Proposed System: Policy & Guardrails Configuration	**COMPLETED AND VERIFIED**	`policies/scaling-policies.yaml` configured with 5 policies (priorities 30–90, hysteresis 5s–30s, cooldown 60s); guardrails clamp replicas between 1 and 10.	None. Policies are complete, valid, and reloaded dynamically.	Keep policies constant during experimental runs.
HPA Collision Detection Guard	**COMPLETED AND VERIFIED**	`controller/strategy_coordinator.py` checks `list_namespaced_horizontal_pod_autoscaler` and blocks scaling if an HPA targets `sample-web-app`.	None. Code exists and is active.	Ensure HPA is deleted/detached when testing KubePilot, and attached when testing Baseline.
Monitoring: Prometheus & cAdvisor	**COMPLETED AND VERIFIED**	`prometheus-server-857bbff68c-nw5s8` running in `monitoring`; port 9090 scrapes `sample-web-app` CPU, memory, and HTTP requests.	None for data collection; metric recording exporter needed for benchmarks.	Port-forward port 9090 during all benchmark runs.
Workload Generator: Tooling Setup (Locust/k6)	**NOT IMPLEMENTED**	Neither `k6` nor `locust` is installed. Python environment contains `requests`, `httpx`, and `aiohttp`, but no load testing framework.	Install `locust` (`pip install locust`) or write a standalone multi-threaded Python traffic generator.	Install `locust` into the virtual environment or deploy a containerized traffic generator.
Workload Pattern 1: Stable Workload	**NOT IMPLEMENTED**	Target workload `sample-web-app` running at 2 replicas.	Workload script simulating constant 20 req/s for 5 minutes; metric capture.	Write Locust scenario `test_stable.py`.
Workload Pattern 2: Gradual Increase	**NOT IMPLEMENTED**	None.	Ramp-up script from 5 req/s to 120 req/s over 8 minutes.	Write Locust scenario `test_gradual.py`.
Workload Pattern 3: Sudden Spike	**NOT IMPLEMENTED**	Mocked in `test_fixes_regression.py` and UI button only.	Step load jumping instantly from 10 req/s to 250 req/s.	Write Locust scenario `test_spike.py`.
Workload Pattern 4: Bursty Workload	**NOT IMPLEMENTED**	None.	Square wave load cycling between 10 req/s and 180 req/s every 60 seconds.	Write Locust scenario `test_bursty.py`.
Workload Pattern 5: Sudden Drop	**NOT IMPLEMENTED**	None.	Step-down script dropping from 200 req/s to 5 req/s to test cooldown & scale-in hysteresis.	Write Locust scenario `test_drop.py`.
Experiment Data Collection & Comparison Harness	**NOT IMPLEMENTED**	Raw `logs/decisions.jsonl` records decisions, but no unified time-series logging exists combining latency, pod CPU, and replica counts for HPA vs KubePilot.	Automated script to poll Prometheus every 2s, export time-series to CSV, and plot comparison charts.	Create `benchmarks/run_experiment.py` to record and plot metrics.

---

6. Overall Progress & Implementation Analysis

Summary Statistics

- **Completed and Verified: 3 items** (Policy configuration, HPA collision guard, Prometheus/cAdvisor monitoring)
- **Implemented but Not Verified: 1 item** (Proposed KubePilot PRAH live execution)
- **Partially Completed: 0 items**
- **Not Implemented: 9 items** (Baseline HPA, Workload generator tool, 5 workload scenarios, Experiment benchmark harness)
- **Blocked: 0 items** *(No blocking infrastructure defects; Docker, Minikube, metrics-server, and Prometheus are all operational)*

Realistic Overall Completion Estimate: **28%**

$$\text{Completion} = \frac{3 \text{ Verified} \times 1.0 + 1 \text{ Implemented} \times 0.6}{13 \text{ Total Items}} = \frac{3.6}{13} \approx 27.7\%$$

Explanation:
The architectural core (the application, Prometheus telemetry pipeline, PRAH policy engine, guardrails, dry-run controller, and web dashboard) is 100% complete and tested. However, **experimentation itself has not started**: zero workload

scripts exist, native HPA is not deployed, no load tests have been executed against the live cluster, and no comparative metrics have been recorded.

---

7. Top Five Priority Tasks

1. **Install Workload Generator Tooling:** Install `locust` (`pip install locust`) or create a standardized Python `asyncio`/`aiohttp` load generator to simulate HTTP traffic against `sample-web-app:8080`.
2. **Implement Native HPA Baseline Manifest:** Create `k8s/baseline-hpa.yaml` configuring native Kubernetes HPA (min: 2, max: 10, target CPU: 75%) to establish the baseline.
3. **Write the 5 Workload Scenarios:** Create parameterized load test profiles (Stable, Gradual, Spike, Bursty, Drop) that can target the port-forwarded `http://localhost:8080` or ClusterIP.
4. **Build the Experiment Logger & Benchmark Harness:** Create a script (`benchmarks/runner.py`) that executes a workload profile, polls Prometheus every 2 seconds for CPU/latency/replicas, and saves a timestamped CSV (`baseline_spike.csv`, `kubepilot_spike.csv`).
5. **Execute Closed-Loop Comparison Runs:** Run the 5 scenarios against Baseline HPA, then disable HPA, run KubePilot with `DRY_RUN=false`, repeat the 5 scenarios, and plot the comparative response time and scaling stability curves.

---

8. Dependencies and Blockers

- **Docker & Minikube: Healthy & Ready.** Cluster is running (`minikube`, Docker driver, 2 CPUs, 4GB RAM, IP `192.168.49.2`).
- **Metrics-Server: Healthy & Ready.** `metrics-server` pod is active in `kube-system`; `kubectl top` functions.
- **Prometheus Scrape Pipeline: Healthy & Ready.** `prometheus-server` scrapes the workload.
- **Workload Generator: Missing Dependency.** Need to install `locust` via `pip install locust` (or use a lightweight Python traffic script).
- **HPA vs KubePilot Mutual Exclusion: Critical Operational Note.** Because `StrategyCoordinator` enforces single-owner scaling, you cannot run KubePilot live scaling while the baseline HPA is active on `sample-web-app`. Tests must be executed sequentially (Baseline Run $\to$ Delete HPA $\to$ Enable KubePilot Live Mode $\to$ Proposed Run).

---

9. Minimum Viable Experimental Plan (MVEP)

To obtain publication-grade, reviewer-ready experimental evidence with minimal friction:

Step 1: Install Locust & Create Scenarios

```
pip install locust
```

Create a single parameterized script `benchmarks/workload_patterns.py` defining:

- `stable`: 20 users constant
- `gradual`: 5 $\to$ 100 users over 6 minutes
- `spike`: 10 $\to$ 200 users instantly
- `bursty`: 10 users $\to$ 150 users $\to$ 10 users in 60s cycles
- `drop`: 150 users $\to$ 5 users instantly

Step 2: Establish the Native HPA Baseline

1. Apply `k8s/baseline-hpa.yaml`:

```
kubectl autoscale deployment sample-web-app --namespace=autoscaling-demo --cpu-percent=75 --min=2 --max=10
```

2. Run the 5 workloads against `http://localhost:8080`.
3. Record scaling latency, pod replica transitions, and CPU in `results/baseline_<pattern>.csv`.

Step 3: Run the Proposed KubePilot System

1. Remove the native HPA to avoid collision rejection:

```
kubectl delete hpa sample-web-app -n autoscaling-demo
```

2. Start KubePilot in live mode:

```
$env:DRY_RUN="false"; python -m controller.main
```

3. Run the identical 5 workloads against `http://localhost:8080`.
4. Record decisions, scaling latency, and replica counts in `results/kubepilot_<pattern>.csv`.

Step 4: Generate Comparison Plots

Run an analysis script (`python benchmarks/plot_results.py`) to generate:

- Replica count vs time (HPA vs KubePilot)
- CPU utilization vs time
- P95 latency and oscillation/flapping count

---

10. Reviewer Claims Guidance

Claims We CAN Safely Present Today (Backed by Verified Evidence)

- "We have built and validated a complete multi-metric autoscaling control plane with a web dashboard, real-time Prometheus telemetry integration, and an extensible PRAH policy engine." *(Verified by passing test suite and running cluster)*
- "The decision engine accurately detects CPU and memory pressure, enforces priority arbitration among competing policies, and applies cooldowns and safety guardrails." *(Verified by `test_policy_engine.py`, `test_guardrails.py`, and `logs/decisions.jsonl`)*
- "The framework enforces strict single-owner controller coordination, actively preventing conflicting mutations when a native Kubernetes HPA is detected on the target workload." *(Verified in `controller/strategy_coordinator.py`)*
- "The control plane separates dry-run preview mode from live cluster mutations, ensuring safe auditing before applying replica changes to the Kubernetes Deployment API." *(Verified in `controller/kubernetes_executor.py`)*

Claims We MUST NOT Make Yet (Not Supported by Evidence)

- "*KubePilot outperforms native Kubernetes HPA in response time and scaling stability.*" — **False today.** No comparative runs have been executed.
- "*We tested the system under 5 distinct synthetic traffic patterns (spike, bursty, etc.).*" — **False today.** No traffic generator has been run; only unit tests and synthetic UI mocks exist.
- "*KubePilot prevented pod flapping during bursty workloads.*" — **False today.** Hysteresis logic is coded and unit-tested with mock timestamps, but real-world oscillation suppression under fluctuating HTTP load has not yet been demonstrated.
- "*KubePilot reduces resource over-provisioning by X%.*" — **False today.** No benchmark numbers exist.